GaiaOps ESG

Testes automatizados com BDD e testes de API

Cucumber (Gherkin) · REST Assured · JSON Schema · GitHub Actions

FIAP · Fase Testes Automatizados

Integrante	RM
João Victor Semente Dias	RM562065

Repositório: https://github.com/JotaDiasDevs/gaiaops-esg-testes

02/11/2026

Sumário

1. Objetivo e contexto ESG
2. Estratégia de testes
3. Cenários BDD em Gherkin
4. Testes de API: status, corpo JSON e contrato
5. Configuração
6. Como executar os testes
7. Evidências da execução local
8. Evidências da execução no GitHub Actions
9. Bug encontrado e desafios
10. Tecnologias e atendimento aos requisitos

1. Objetivo e contexto ESG

O **GaiaOps ESG** é a API de gestão ESG do município fictício de **Vila Nova do Horizonte**, desenvolvida em Java 21 + Spring Boot 3.5 + MongoDB nas atividades anteriores. Nesta fase, a API recebeu uma suíte de testes automatizados que valida suas funcionalidades críticas e, com elas, a contribuição da aplicação para cada pilar ESG.

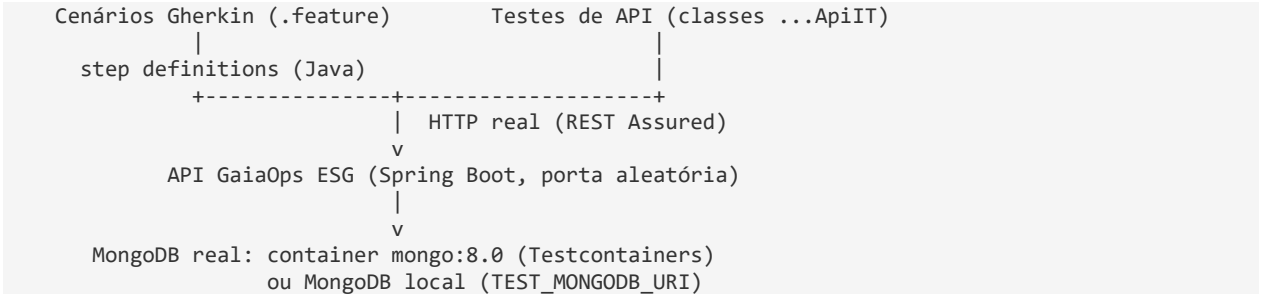
Exemplo do enunciado	Como aparece no GaiaOps	Onde é testado
Eficiência energética e sustentabilidade	Medições de energia e água, alertas de perdas na rede; contrato do painel ESG	02_ambiental_alertas_sensores.feature, MedicaoApiIT, IndicadorApiIT
Gestão de resíduos e reciclagem	Coleta seletiva, destinação e taxa de desvio de aterro (83.702 kg, 49% desviados)	03_ambiental_gestao_residuos.feature, ResiduoApiIT
Inclusão e diversidade	Programas sociais, recurso de acessibilidade para pessoa com deficiência, beneficiários sem dados pessoais (LGPD)	04_social_inclusao_programas.feature, ProgramaSocialApiIT, contrato programa-social
Governança e compliance ambiental	Integridade do cadastro, licenças vencidas, rastreabilidade da versão implantada	01_governanca_cadastro_equipamentos.feature, 05_governanca_conformidade.feature

2. Estratégia de testes

Camada	Ferramentas	O que valida	Quantidade
Cenários BDD	Cucumber 7, Gherkin em português, REST Assured	Comportamento de negócio ponta a ponta: caminhos felizes e de falha	23 cenários (9 negativos)
Testes de API	REST Assured, JSON Schema Validator	Todas as rotas: status code, corpo JSON e contrato	46 testes
Testes unitários	JUnit 5, Mockito, MockMvc	Regras de alerta e camada web isoladas	17 testes

2.1 Arquitetura da execução

Figura 1 - Cenários BDD e testes de API usam a aplicação completa e um banco real



- **Comunicação real entre os serviços.** A API sobe completa num servidor HTTP em porta aleatória, e todas as chamadas passam pela rede, pelo Spring MVC, pela serialização JSON e pelo MongoDB. Não há mocks entre a API e o banco.
- **Um contexto para tudo.** Cenários BDD e testes de API herdam da mesma classe (BaseIntegracao). O Spring reaproveita uma única aplicação e um único banco durante toda a execução.
- **Dados isolados.** Antes de cada cenário e de cada teste, a base volta aos 100 documentos da carga inicial. Nenhum teste depende da ordem nem do que outro gravou.
- **Tempo controlado.** O relógio da aplicação é substituído por um relógio de teste (01/10/2026). Regras como "licença vencida há 16 dias" têm resultado estável em qualquer data, e um cenário pode ajustar a data ("Dado que a data de referência é 01/09/2027").

3. Cenários BDD em Gherkin

Os cenários foram escritos em Gherkin, em português (`# language: pt`), do ponto de vista de quem usa a plataforma: gestor de governança, vigilância ambiental, coordenação da coleta seletiva, assistência social e auditoria. Cada funcionalidade combina **caminhos felizes** (`@positivo`) e **caminhos de falha** (`@negativo`), e as tags também indicam o pilar ESG (`@ambiental`, `@social`, `@governanca`).

São **5 funcionalidades** e **20 cenários escritos**. Na execução, os *Esquemas do Cenário* se expandem em um cenário por linha de exemplos, totalizando 23 cenários executados.

3.1 Cadastro íntegro dos equipamentos públicos

Tipo	Cenário
positivo	Cadastrar uma nova escola municipal
negativo	Recusar cadastro que fere as regras de integridade (<motivo>) (Esquema do Cenário)
negativo	Impedir dois equipamentos com o mesmo código
positivo	Desativar um equipamento preserva o histórico

Listagem - src/test/resources/features/01_governanca_cadastro_equipamentos.feature

```
# language: pt
@governanca @equipamentos
Funcionalidade: Cadastro íntegro dos equipamentos públicos
  Como gestor de governança da prefeitura de Vila Nova do Horizonte
  Quero que o cadastro de equipamentos públicos só aceite dados válidos
  Para que todos os indicadores ESG da cidade sejam calculados sobre dados confiáveis

  Contexto:
    Dado que a base ESG do município está carregada com 12 equipamentos

  @positivo
  Cenário: Cadastrar uma nova escola municipal
    Quando o gestor cadastra o equipamento:
      | codigo             | EQP-ESC-002           |
      | nome               | EMEF Jardim das Flores |
      | tipo               | escola                |
      | distrito           | Morro Alto            |
      | populacao_atendida | 640                   |
    Então a API responde com o status 201
    E a resposta segue o contrato "equipamento"
    E o equipamento "EQP-ESC-002" pode ser consultado e está ativo
    E o painel ESG conta 13 equipamentos ativos de 13

  @negativo
  Esquema do Cenário: Recusar cadastro que fere as regras de integridade (<motivo>)
    Quando o gestor cadastra o equipamento com código "<codigo>", nome "<nome>" e tipo "<tipo>"
    Então a API responde com o status 400
    E a resposta segue o contrato "problema-validacao"
    E a resposta aponta erro no campo "<campo>"
    E o painel ESG conta 12 equipamentos ativos de 12

    Exemplos:
      | motivo                | codigo      | nome      | tipo     | campo  |
      | código fora do padrão | escola-2    | EMEF Nova | escola   | codigo |
      | tipo inexistente      | EQP-SHP-001 | Shopping  | shopping | tipo   |
      | nome curto demais     | EQP-ESC-003 | AB        | escola   | nome   |

  @negativo
  Cenário: Impedir dois equipamentos com o mesmo código
    Quando o gestor cadastra o equipamento com código "EQP-ESC-001", nome "Outra escola" e tipo
"escola"
    Então a API responde com o status 409
    E a resposta segue o contrato "problema"
```

```
    E a mensagem de erro contém "EQP-ESC-001"

  @positivo @rastreabilidade
  Cenário: Desativar um equipamento preserva o histórico
    Quando o gestor desativa o equipamento "EQP-ILU-001"
    Então a API responde com o status 204
    E o equipamento "EQP-ILU-001" continua registrado, porém inativo
    E o painel ESG conta 11 equipamentos ativos de 12
```

3.2 Alertas automáticos da telemetria urbana

Tipo	Cenário
positivo	Classificar o alerta de qualidade do ar pelo IQAr (<situacao>) (Esquema do Cenário)
positivo	Leitura dentro dos limites é registrada sem alerta
positivo	Córrego acima da cota de alerta dispara alerta crítico e entra no painel
negativo	Equipamento desativado não recebe leituras
negativo	Leitura de sensor de equipamento inexistente é recusada

Listagem - src/test/resources/features/02_ambiental_alertas_sensores.feature

```
# language: pt
@ambiental @medicoes
Funcionalidade: Alertas automáticos da telemetria urbana
  Como equipe de vigilância ambiental do município
  Quero que leituras fora dos limites legais gerem alertas automaticamente
  Para agir rápido em situações que afetam a saúde da população

  Contexto:
    Dado que a base ESG do município está carregada com 12 equipamentos

  @positivo
  Esquema do Cenário: Classificar o alerta de qualidade do ar pelo IQAr (<situacao>)
    Quando o sensor do equipamento "EQP-PRA-002" envia uma leitura de qualidade do ar com IQAr
<iqar>
    Então a API responde com o status 201
    E a resposta segue o contrato "medicao"
    E a leitura gera um alerta de nível "<nivel>"

    Exemplos:
      | situacao          | iqar | nivel   |
      | ar ruim           | 151  | critico |
      | ar moderado       | 95   | medio   |

  @positivo
  Cenário: Leitura dentro dos limites é registrada sem alerta
    Quando o sensor do equipamento "EQP-PRA-002" envia uma leitura de qualidade do ar com IQAr 60
    Então a API responde com o status 201
    E a resposta segue o contrato "medicao"
    E a leitura é registrada sem alerta

  @positivo
  Cenário: Córrego acima da cota de alerta dispara alerta crítico e entra no painel
    Quando o sensor do equipamento "EQP-PRA-001" envia uma leitura de "hidrologia" com:
      | bloco      | campo            | valor |
      | leitura    | nivel_cm         | 251   |
      | referencia | nivel_atencao_cm | 180   |
      | referencia | nivel_alerta_cm  | 240   |
    Então a API responde com o status 201
    E a leitura gera um alerta de nível "critico"
    E o alerta aparece entre os alertas abertos de nível "critico"
    E o painel ESG passa a ter 7 alertas abertos

  @negativo
  Cenário: Equipamento desativado não recebe leituras
```

```
    Dado que o equipamento "EQP-TER-001" foi desativado
    Quando o sensor do equipamento "EQP-TER-001" envia uma leitura de "ruido" com:
      | bloco   | campo   | valor |
      | leitura | laeq_db | 60    |
    Então a API responde com o status 422
    E a resposta segue o contrato "problema"
    E a mensagem de erro contém "desativado"

  @negativo
  Cenário: Leitura de sensor de equipamento inexistente é recusada
    Quando o sensor do equipamento "EQP-XYZ-999" envia uma leitura de qualidade do ar com IQAr 80
    Então a API responde com o status 404
    E a resposta segue o contrato "problema"
    E o painel ESG passa a ter 6 alertas abertos
```

3.3 Gestão de resíduos e reciclagem

Tipo	Cenário
positivo	Painel ESG calcula a taxa de desvio de aterro
positivo	Resumo por categoria mostra para onde vai cada resíduo
negativo	Consulta de categoria sem coletas não inventa dados

Listagem - src/test/resources/features/03_ambiental_gestao_residuos.feature

```
# language: pt
@ambiental @residuos
Funcionalidade: Gestão de resíduos e reciclagem
  Como coordenador da coleta seletiva
  Quero acompanhar quanto dos resíduos coletados deixa de ir para o aterro
  Para cumprir a Política Nacional de Resíduos Sólidos e valorizar as cooperativas

  @positivo
  Cenário: Painel ESG calcula a taxa de desvio de aterro
    Quando consulto o painel ESG do município
    Então a API responde com o status 200
    E a resposta segue o contrato "painel-esg"
    E o total de resíduos coletados é de 83702 kg
    E a taxa de desvio de aterro é de 49,0%

  @positivo
  Cenário: Resumo por categoria mostra para onde vai cada resíduo
    Quando consulto o resumo de resíduos por categoria
    Então a API responde com o status 200
    E a resposta segue o contrato "residuos-resumo"
    E a categoria "rejeito" com destinação "aterro_sanitario" soma 42200 kg
    E a categoria "organico" com destinação "compostagem" soma 6712 kg

  @negativo
  Cenário: Consulta de categoria sem coletas não inventa dados
    Quando consulto as coletas da categoria "radioativo"
    Então a API responde com o status 200
    E a resposta segue o contrato "residuos"
    E nenhuma coleta é retornada
```

3.4 Inclusão, diversidade e privacidade nos programas sociais

Tipo	Cenário
positivo	Programa de qualificação garante recurso de acessibilidade a beneficiário com deficiência
positivo	Nenhum programa expõe dados pessoais dos beneficiários
positivo	Painel social consolida beneficiários e execução orçamentária

negativo	Consulta de programa inexistente

Listagem - src/test/resources/features/04_social_inclusao_programas.feature

```
# language: pt
@social @programas
Funcionalidade: Inclusão, diversidade e privacidade nos programas sociais
  Como gestora da Secretaria de Assistência Social
  Quero acompanhar os programas e seus beneficiários sem expor dados pessoais
  Para promover inclusão respeitando a LGPD

  @positivo @inclusao
  Cenário: Programa de qualificação garante recurso de acessibilidade a beneficiário com
deficiência
    Quando consulto o programa social "PRG-TR-001"
    Então a API responde com o status 200
    E a resposta segue o contrato "programa-social"
    E o programa atende um beneficiário com deficiência com o recurso "interprete de libras"

  @positivo @lgpd
  Cenário: Nenhum programa expõe dados pessoais dos beneficiários
    Quando consulto todos os programas sociais
    Então a API responde com o status 200
    E a resposta segue o contrato "programas-sociais"
    E nenhum beneficiário expõe os campos "nome, cpf, rg, email, telefone"
    E todo beneficiário é identificado apenas por protocolo

  @positivo
  Cenário: Painel social consolida beneficiários e execução orçamentária
    Quando consulto o painel ESG do município
    Então o painel social soma 25 beneficiários em 11 programas ativos
    E a execução orçamentária dos programas é de 58,5%

  @negativo
  Cenário: Consulta de programa inexistente
    Quando consulto o programa social "PRG-XX-999"
    Então a API responde com o status 404
    E a resposta segue o contrato "problema"
    E a mensagem de erro contém "PRG-XX-999"
```

3.5 Conformidade regulatória e rastreabilidade

Tipo	Cenário
positivo	Licença vencida é sinalizada mesmo com a situação desatualizada no cadastro
positivo	Licença de operação entra no radar de renovação perto do vencimento
positivo	Versão implantada é rastreável
negativo	Prazo negativo não esconde licenças já vencidas

Listagem - src/test/resources/features/05_governanca_conformidade.feature

```
# language: pt
@governanca @conformidade
Funcionalidade: Conformidade regulatória e rastreabilidade
  Como auditor de compliance ambiental
  Quero que licenças vencidas sejam sinalizadas e que cada versão implantada seja rastreável
  Para que a prefeitura não opere equipamentos fora da lei e saiba o que está em produção

  @positivo
  Cenário: Licença vencida é sinalizada mesmo com a situação desatualizada no cadastro
    Dado que a data de referência é 01/10/2026
    Quando consulto as licenças que vencem nos próximos 180 dias
    Então a API responde com o status 200
    E a resposta segue o contrato "licencas-vencimento"
    E a licença "LP-11.204/2025" aparece como vencida há 16 dias
```

```
    E a licença "LO-45.678/2023" não aparece na lista

  @positivo
  Cenário: Licença de operação entra no radar de renovação perto do vencimento
    Dado que a data de referência é 01/09/2027
    Quando consulto as licenças que vencem nos próximos 180 dias
    Então a API responde com o status 200
    E a licença "LO-45.678/2023" aparece a vencer em 90 dias

  @positivo @rastreabilidade
  Cenário: Versão implantada é rastreável
    Quando consulto as informações da versão implantada
    Então a API responde com o status 200
    E a resposta segue o contrato "info"
    E a resposta identifica a aplicação "GaiaOps ESG" na versão "1.0.0"

  @negativo
  Cenário: Prazo negativo não esconde licenças já vencidas
    Dado que a data de referência é 01/10/2026
    Quando consulto as licenças que vencem nos próximos -30 dias
    Então a API responde com o status 200
    E a licença "LP-11.204/2025" aparece como vencida há 16 dias
```

3.6 Como as frases viram código (step definitions)

Cada frase do Gherkin corresponde a um método Java anotado. O "Quando" faz a chamada HTTP real e guarda a resposta no contexto do cenário; os "Então" verificam essa resposta. Trecho de `MedicaoSteps.java`:

Listagem - step definitions de medições

```
    @Quando("o sensor do equipamento {string} envia uma leitura de qualidade do ar com IQAr
{int}")
    public void oSensorEnviaLeituraDeQualidadeDoAr(String equipamento, int iqar) {
        enviar(equipamento, "qualidade_ar", Map.of("indice_iqar", iqar), null);
    }

        @Quando("o sensor do equipamento {string} envia uma leitura de {string} com:")
    public void oSensorEnviaLeitura(String equipamento, String tipo, List<Map<String, String>>
linhas) {
        Map<String, Object> leitura = new LinkedHashMap<>();
        Map<String, Object> referencia = new LinkedHashMap<>();
        for (Map<String, String> linha : linhas) {
            Map<String, Object> bloco = "referencia".equals(linha.get("bloco")) ? referencia :
leitura;
            bloco.put(linha.get("campo"), Double.valueOf(linha.get("valor")));
        }
        enviar(equipamento, tipo, leitura, referencia.isEmpty() ? null : referencia);
    }

    @Entao("a leitura gera um alerta de nível {string}")
    public void aLeituraGeraUmAlertaDeNivel(String nivel) {
        contexto.validar()
                .body("alerta.nivel", equalTo(nivel))
                .body("alerta.reconhecido", equalTo(false));
    }
```

4. Testes de API: status, corpo JSON e contrato

Os testes de API ficam nas classes terminadas em `ApiIT` (uma por recurso). Cada teste faz uma chamada HTTP real e verifica o **status code**, o **corpo JSON** (valores, quantidades, ordenação e efeitos de uma gravação nas consultas seguintes) e o **contrato JSON Schema** da resposta.

Rota	Métodos	Status testados	Contratos
/api/equipamentos	GET (filtros, limite), POST	200, 201, 400, 409, 422	equipamentos, equipamento, problema, problema-validacao
/api/equipamentos/{codigo}	GET, PUT, DELETE	200, 204, 400, 404	equipamento, problema, problema-validacao
/api/medicoes	GET (filtros, limite), POST	200, 201, 400, 404, 422	medicoes, medicao, problema, problema-validacao
/api/medicoes/alertas	GET (nível, abertos)	200	medicoes
/api/residuos	GET (categoria)	200	residuos
/api/residuos/resumo	GET	200	residuos-resumo
/api/programas-sociais	GET (eixo)	200	programas-sociais
/api/programas-sociais/{codigo}	GET	200, 404	programa-social, problema
/api/conformidade	GET (tipo)	200	conformidades
/api/conformidade/licencas/vencimentos	GET (dias)	200	licencas-vencimento
/api/indicadores	GET	200	painel-esg
/actuator/health, /readiness, /liveness, /actuator/info	GET	200	health, info
/v3/api-docs, /, rota inexistente	GET	200, 404	problema

Garantia de que todas as APIs estão cobertas: o teste `PlataformaApiIT.cobreTodasAsRotas` lê a documentação OpenAPI gerada pela própria aplicação e compara as rotas com a lista coberta pela suíte. Um endpoint novo sem teste faz esse teste falhar.

4.1 Exemplo de teste de API

Listagem - MedicaoApiIT: status, corpo JSON e contrato numa só verificação

```
        @Test
        @DisplayName("201: IQAr 151 gera alerta critico; data e competencia vem do relogio")
        void registraComAlerta() {
            api().body("""
                            { "equipamento_id": "EQP-PRA-002", "tipo": "qualidade_ar",
                              "sensor": { "id": "SEN-AR-004" },
                              "leitura": { "mp10_ug_m3": 95, "indice_iqar": 151 } }""")
                    .post("/api/medicoes").then()
                    .statusCode(201)
                    .body(contrato("medicao"))
                    .body("alerta.nivel", equalTo("critico"))
                    .body("alerta.regra", containsString("130"))
                    .body("alerta.valor_medido", equalTo(151))
                    .body("data", startsWith("2026-10-01T12:00:00"))
                    .body("competencia", equalTo("2026-10"));
        }
```

4.2 Contratos JSON Schema

Contrato	Descreve
conformidade.json	Registro de governanca municipal
conformidades.json	Lista de registros de governanca
equipamento.json	Equipamento publico do municipio
equipamentos.json	Lista de equipamentos
health.json	Saude da aplicacao (Actuator)
info.json	Informacoes da versao implantada (rastreabilidade)
licencas-vencimento.json	Licencas vencidas ou a vencer
medicao.json	Medicao de sensor urbano
medicoes.json	Lista de medicoes
painel-esg.json	Painel ESG do municipio
problema-validacao.json	Erro de validacao: Problem Details + mapa campo -> mensagem
problema.json	Problem Details (RFC 9457) - formato padrao de erro da API
programa-social.json	Programa social do municipio
programas-sociais.json	Lista de programas sociais
residuo.json	Coleta de residuos
residuos-resumo.json	Peso coletado por categoria e destinacao
residuos.json	Lista de coletas de residuos

Destaques: o contrato do painel ESG é **fechado** (`additionalProperties: false`), porque consumidores dependem exatamente daqueles indicadores. O contrato de equipamento exige os campos de governança e **aceita atributos livres** por tipo, preservando o esquema flexível do MongoDB. O contrato de programa social **proíbe dados pessoais** dos beneficiários, transformando uma regra da LGPD em verificação automática:

Listagem - programa-social.json: protocolo pseudonimizado e campos pessoais proibidos

```
"beneficiarios": {
      "type": "array",
      "items": {
        "type": "object",
        "required": [
          "protocolo",
          "situacao"
        ],
        "properties": {
          "protocolo": {
            "type": "string",
            "pattern": "^[A-Z]{3}-[A-Z0-9-]+$"
          },
          "situacao": {
            "type": "string",
            "minLength": 3
          },
          "pcd": {
            "type": "boolean"
          }
```

```
        },
        "not": {
          "anyOf": [
            {
              "required": [
                "nome"
              ]
            },
            {
              "required": [
                "cpf"
              ]
            },
            {
              "required": [
                "rg"
              ]
            },
            {
              "required": [
                "email"
              ]
            },
            {
              "required": [
                "telefone"
              ]
            }
          ]
        }
      }
    }
  }
}
```

5. Configuração

5.1 Dependências de teste (pom.xml)

Listagem - pom.xml: Cucumber, REST Assured e JSON Schema

```
        <!-- BDD: cenarios em Gherkin executados pelo Cucumber sobre a JUnit Platform -->
        <dependency>
            <groupId>io.cucumber</groupId>
            <artifactId>cucumber-java</artifactId>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>io.cucumber</groupId>
            <artifactId>cucumber-spring</artifactId>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>io.cucumber</groupId>
            <artifactId>cucumber-junit-platform-engine</artifactId>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>org.junit.platform</groupId>
            <artifactId>junit-platform-suite</artifactId>
            <scope>test</scope>
        </dependency>

        <!-- Testes de API: chamadas HTTP reais + validacao de contrato com JSON Schema -->
        <dependency>
            <groupId>io.rest-assured</groupId>
            <artifactId>rest-assured</artifactId>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>io.rest-assured</groupId>
            <artifactId>json-schema-validator</artifactId>
            <scope>test</scope>
        </dependency>
```

Listagem - pom.xml: versões e propriedades

```
        <cucumber.version>7.34.9</cucumber.version>
        <!-- O Cucumber 7.34 e compilado contra o JUnit 5.14; o Spring Boot 3.5 traz o 5.12. -->
        <junit-jupiter.version>5.14.2</junit-jupiter.version>
        <!-- Permite rodar so os testes de integracao: ./mvnw verify -DskipUnitTests=true -->
        <skipUnitTests>false</skipUnitTests>
```

5.2 Runner do Cucumber

Listagem - CucumberIT.java: features, glue e relatórios

```
@Suite
@IncludeEngines("cucumber")
@SelectClasspathResource("features")
@ConfigurationParameter(key = GLUE_PROPERTY_NAME, value = "br.com.fiap.gaiaops.bdd")
@ConfigurationParameter(key = PLUGIN_PROPERTY_NAME, value = "pretty,"
        + " html:target/cucumber-reports/cucumber.html,"
        + " json:target/cucumber-reports/cucumber.json,"
        + " junit:target/cucumber-reports/cucumber.xml")
@ConfigurationParameter(key = PLUGIN_PUBLISH_QUIET_PROPERTY_NAME, value = "true")
public class CucumberIT {
}
```

5.3 Ambiente de integração comum

Listagem - BaseIntegracao.java: API em porta aleatória, MongoDB real, relógio e restauração da base

```
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
@Import(BaseIntegracao.ConfiguracaoDeTeste.class)
public abstract class BaseIntegracao {

    static {
        RestAssured.config = RestAssuredConfig.config()
                .jsonConfig(JsonConfig.jsonConfig().numberReturnType(NumberReturnType.DOUBLE));
        RestAssured.enableLoggingOfRequestAndResponseIfValidationFails();
    }

    @DynamicPropertySource
    static void mongoDb(DynamicPropertyRegistry registro) {
        registro.add("spring.data.mongodb.uri", MongoDeTeste::uri);
    }

    @TestConfiguration(proxyBeanMethods = false)
    static class ConfiguracaoDeTeste {

                @Bean
        @Primary
        RelogioDeTeste relogioDeTeste() {
            return new RelogioDeTeste();
        }

        @Bean
        BaseDeDadosDeTeste baseDeDadosDeTeste(MongoTemplate mongo, CargaInicial cargaInicial) {
            return new BaseDeDadosDeTeste(mongo, cargaInicial);
        }
    }
}
```

Listagem - MongoDeTeste.java: container descartável ou MongoDB local

```
    public static synchronized String uri() {
        String externa = System.getenv(VARIAVEL);
        if (externa != null && !externa.isBlank()) {
            exigirBancoDescartavel(externa);
            return externa;
        }
        if (container == null) {
            container = new MongoDBContainer("mongo:8.0");
            container.start();
        }
        return container.getReplicaSetUrl("gaiaops_testes");
    }
```

5.4 Pipeline de CI

Listagem - .github/workflows/testes.yml

```
jobs:

  testes:
    name: Testes automatizados
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v7

      - name: Java ${{ env.JAVA_VERSION }} (Temurin) com cache do Maven
        uses: actions/setup-java@v6
        with:
          distribution: temurin
          java-version: ${{ env.JAVA_VERSION }}
          cache: maven

      - name: 1. Testes unitarios (JUnit 5 + Mockito)
        run: ./mvnw -B -ntp test

      # Sobe a API completa em porta aleatoria e um MongoDB 8.0 em container;
      # os cenarios Gherkin e os testes de API fazem chamadas HTTP reais.
```

```
      - name: 2. Cenarios BDD (Cucumber) e testes de API (REST Assured + JSON Schema)
        run: ./mvnw -B -ntp verify -DskipUnitTests=true

      - name: 3. Resumo da execucao
        if: always()
        run: python3 .github/scripts/resumo_testes.py

      - name: Publicar relatorio do Cucumber (HTML)
        if: always()
        uses: actions/upload-artifact@v7
        with:
          name: relatorio-cucumber
          path: target/cucumber-reports/
          retention-days: 30

      - name: Publicar relatorios JUnit e cobertura (JaCoCo)
        if: always()
        uses: actions/upload-artifact@v7
        with:
          name: relatorios-junit-e-cobertura
          path: |
            target/surefire-reports/
            target/failsafe-reports/
            target/site/jacoco/
          retention-days: 30

  imagem:
    name: Build da imagem Docker
    needs: testes
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v7

      - name: Preparar Docker Buildx
        uses: docker/setup-buildx-action@v4

      - name: Build da imagem (sem publicar)
        uses: docker/build-push-action@v7
        with:
          context: .
          push: false
          tags: gaiaops-esg:${{ github.sha }}
          cache-from: type=gha
          cache-to: type=gha,mode=max
```

6. Como executar os testes

Pré-requisito: JDK 21. O Maven vem embutido no projeto (Maven Wrapper). Para o banco de dados, basta **Docker** (o MongoDB sobe sozinho em container) **ou** um **MongoDB instalado**.

Objetivo	Comando
Tudo, com Docker (igual ao pipeline)	`./mvnw verify`
Tudo, com MongoDB local (Linux/macOS/Git Bash)	`export TEST_MONGODB_URI=mongodb://localhost:27017/gaiaops_testes` e `./mvnw verify`
Tudo, com MongoDB local (PowerShell)	`$env:TEST_MONGODB_URI = "mongodb://localhost:27017/gaiaops_testes"` e `.\mvnw.cmd verify`
Só testes unitários	`./mvnw test`
Só cenários BDD e testes de API	`./mvnw verify -DskipUnitTests=true`
Só os cenários BDD	`./mvnw verify -DskipUnitTests=true -Dit.test=CucumberIT`
Só cenários de uma tag	`./mvnw verify -DskipUnitTests=true -Dit.test=CucumberIT "-Dcucumber.filter.tags=@negativo"`
Só os testes de uma API	`./mvnw verify -DskipUnitTests=true -Dit.test=MedicaoApiIT`

Por segurança, no modo MongoDB local o nome do banco precisa conter "teste": os testes apagam e recarregam as collections a cada cenário, e essa trava impede apontá-los para uma base com dados reais.

Relatório gerado	Conteúdo
target/cucumber-reports/cucumber.html	Relatório navegável do Cucumber: cenários, passos, resultado e tempo
target/cucumber-reports/cucumber.json e .xml	Mesmo resultado para ferramentas de CI
target/failsafe-reports e surefire-reports	Resultados JUnit dos testes de API e unitários
target/site/jacoco/index.html	Cobertura de código da aplicação

7. Evidências da execução local

Execução completa (`./mvnw clean verify`) em uma estação Windows 11 com JDK 21 e MongoDB 8 local, no modo `TEST_MONGODB_URI=mongodb://localhost:27017/gaiaops_testes`. As tabelas abaixo foram geradas pelo script de resumo a partir dos relatórios reais dessa execução.

Tipo	Ferramentas	Aprovados	Falhas	Ignorados
Cenarios BDD (Gherkin)	Cucumber + REST Assured	23	0	0
Testes de API	REST Assured + JSON Schema	46	0	0
Testes unitarios	JUnit 5 + Mockito + MockMvc	17	0	0

Cobertura de linhas da aplicação (JaCoCo): **97,6%**

Listagem - saída real do Maven (docs/evidencias/execucao-local-mvn-verify.log)

```
Tests run: 8, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 6.222 s -- in
br.com.fiap.gaiaops.equipamento.EquipamentoControllerTest
Tests run: 9, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.186 s -- in
br.com.fiap.gaiaops.medicao.RegrasDeAlertaTest
Tests run: 17, Failures: 0, Errors: 0, Skipped: 0
Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 16.31 s -- in API
/api/conformidade
Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.524 s -- in DELETE
/api/equipamentos/{codigo}
Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.838 s -- in PUT
/api/equipamentos/{codigo}
Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.260 s -- in POST
/api/equipamentos
Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.373 s -- in GET
/api/equipamentos/{codigo}
Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.567 s -- in GET
/api/equipamentos
Tests run: 0, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 3.583 s -- in API
/api/equipamentos
Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.639 s -- in API /api/indicadores
Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.004 s -- in POST /api/medicoes
Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.420 s -- in GET
/api/medicoes/alertas
Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.611 s -- in GET /api/medicoes
Tests run: 0, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 2.048 s -- in API /api/medicoes
Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.998 s -- in APIs de plataforma
(Actuator, OpenAPI e pagina inicial)
Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.852 s -- in API /api/programas-
sociais
Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.569 s -- in API /api/residuos
Tests run: 23, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 6.006 s -- in
br.com.fiap.gaiaops.bdd.CucumberIT
Tests run: 69, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS
Total time:  51.067 s
```

7.1 Relatório do Cucumber

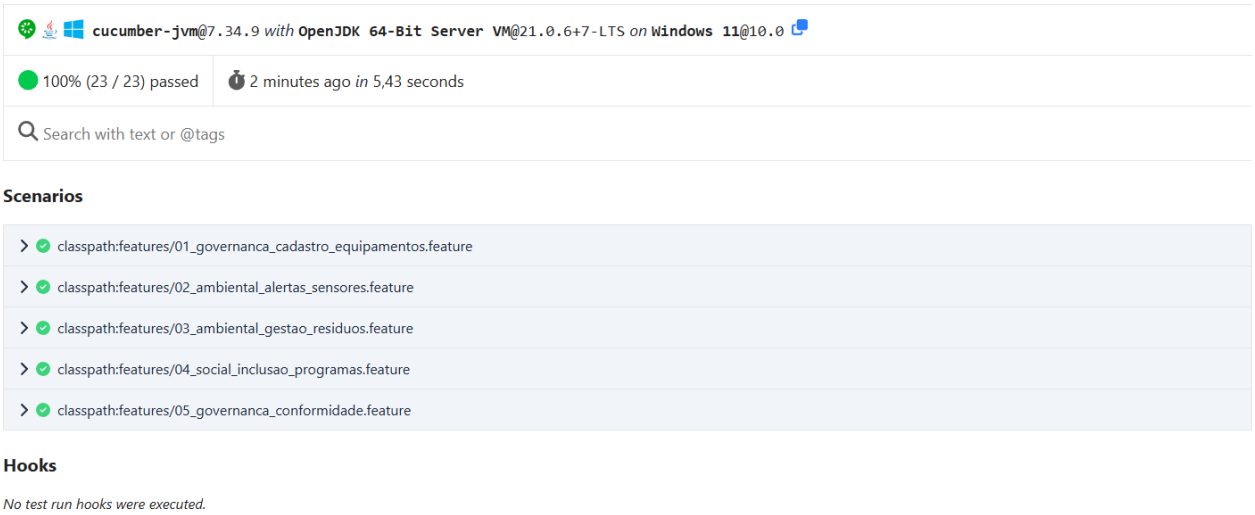


Figura - Relatório do Cucumber: Visão geral: 100% (23/23) dos cenários aprovados

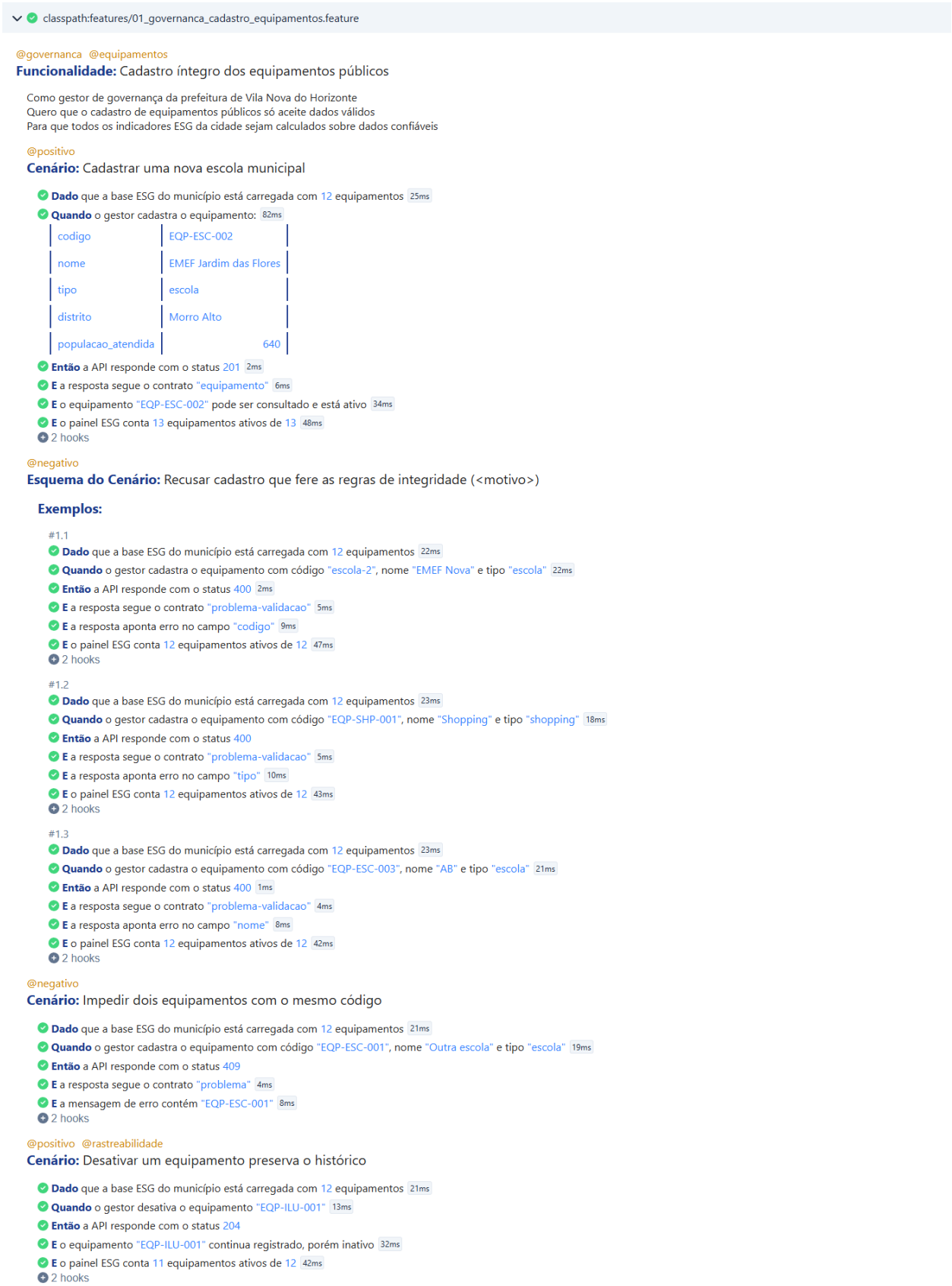


Figura - Relatório do Cucumber: Cadastro íntegro dos equipamentos públicos

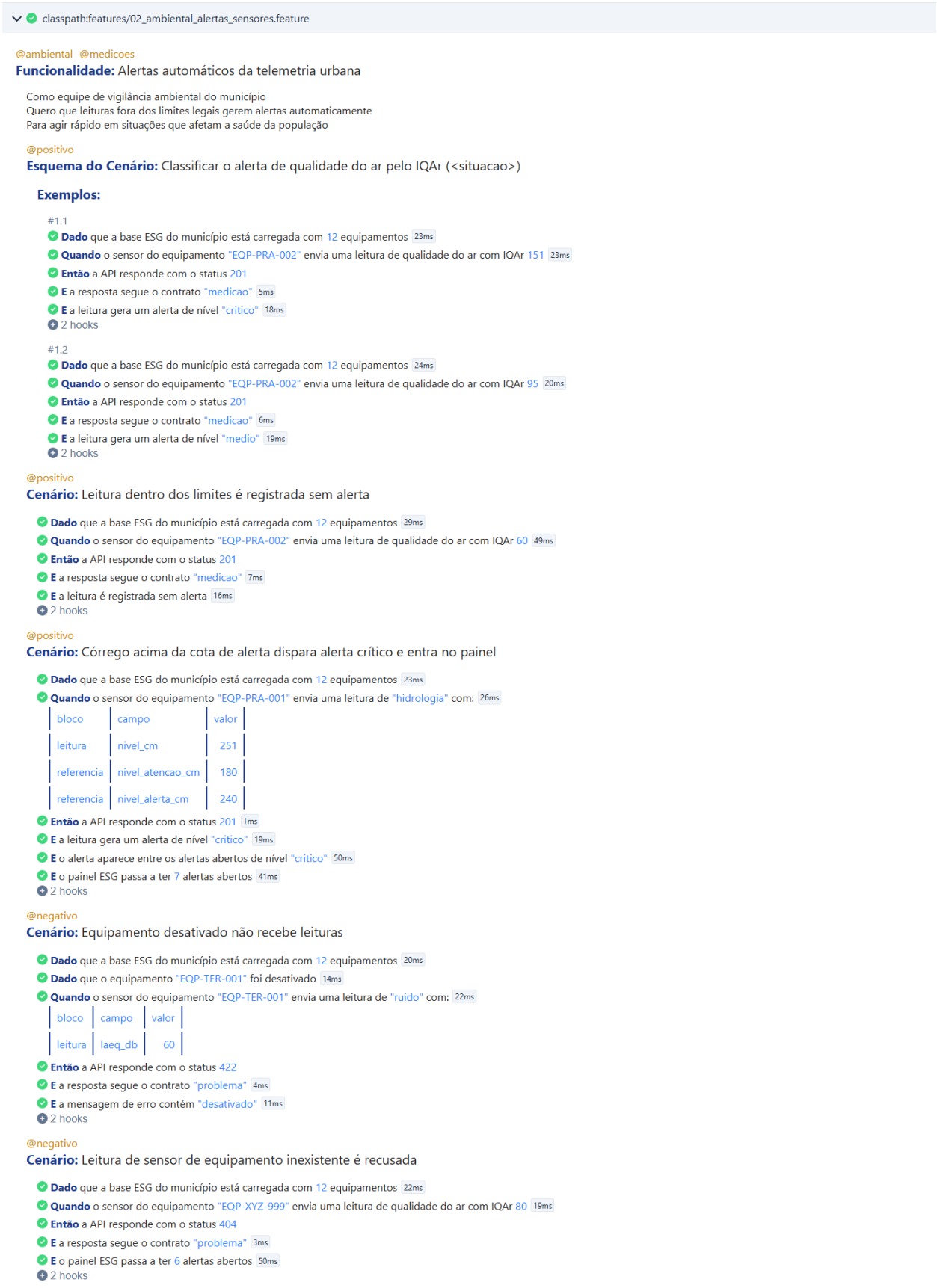


Figura - Relatório do Cucumber: Alertas automáticos da telemetria urbana

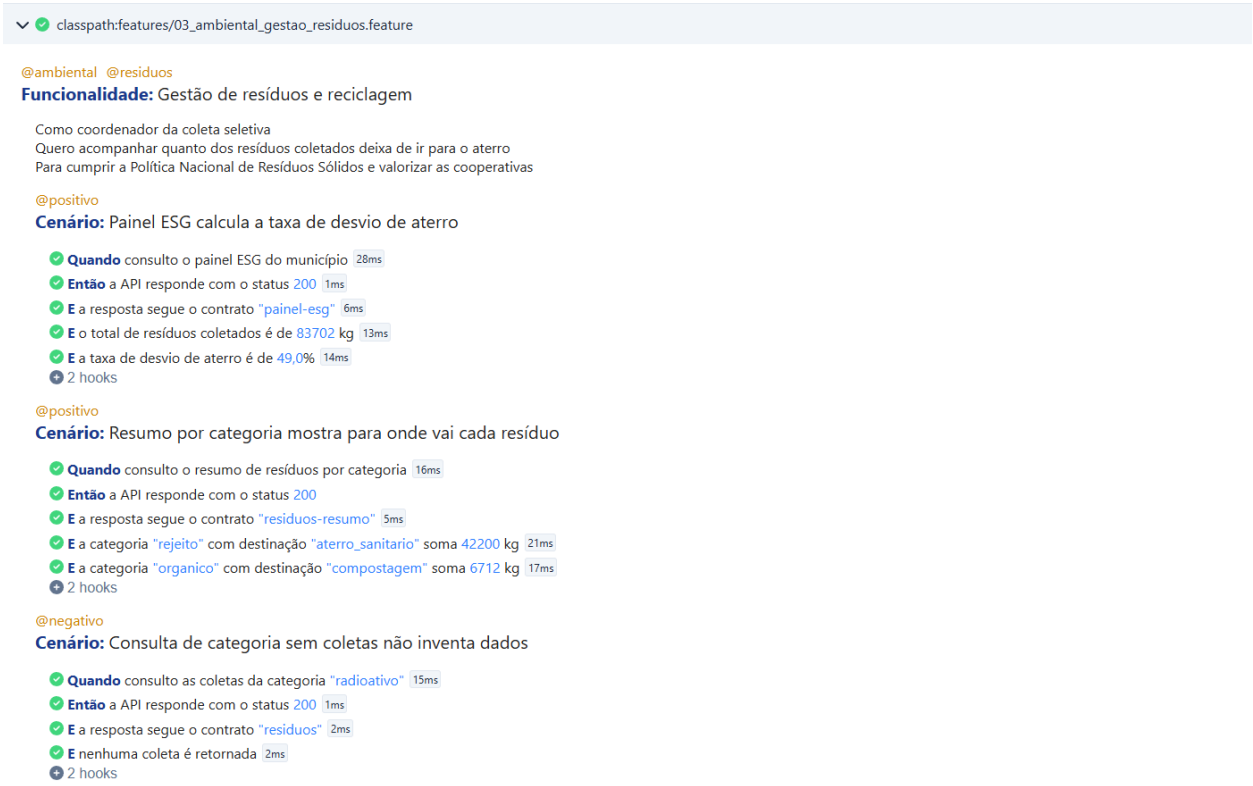


Figura - Relatório do Cucumber: Gestão de resíduos e reciclagem

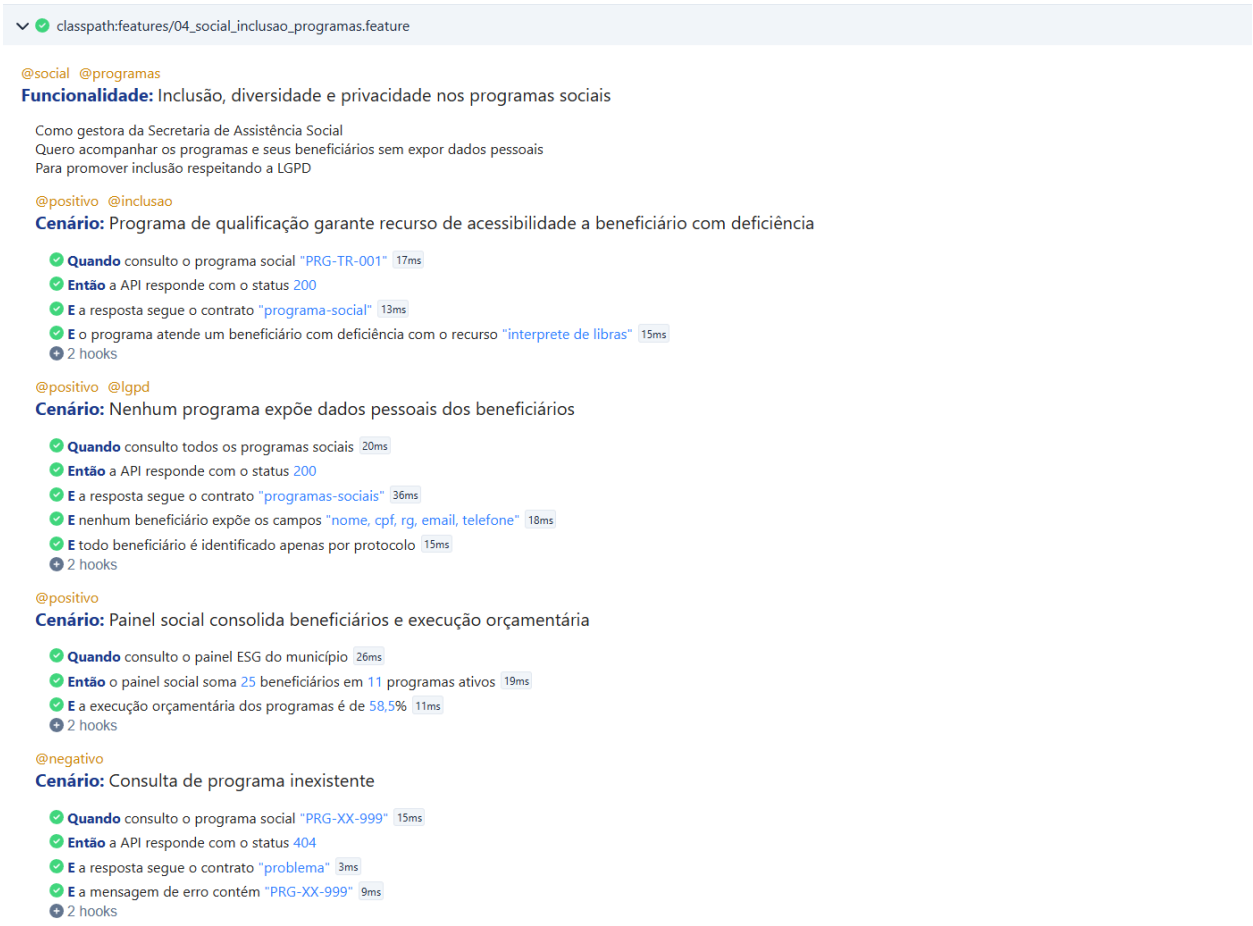


Figura - Relatório do Cucumber: Inclusão, diversidade e privacidade

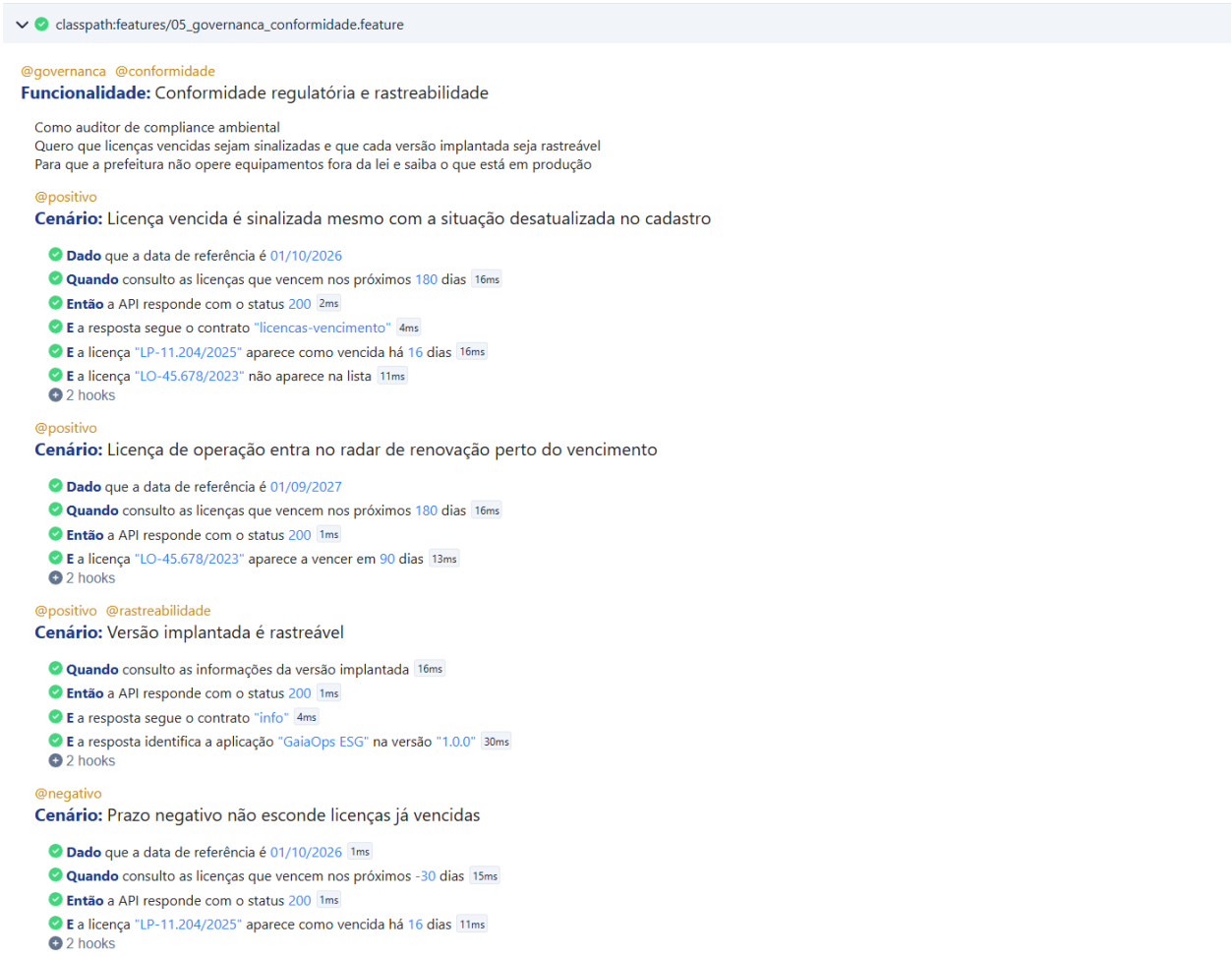


Figura - Relatório do Cucumber: Conformidade regulatória e rastreabilidade

8. Evidências da execução no GitHub Actions

O workflow `Testes automatizados` roda a cada push na branch main e sob demanda. A execução abaixo usa um runner Linux do GitHub, com o MongoDB 8.0 em container criado pelo Testcontainers.

PRINT PENDENTE: docs/prints/ci-01-execucao.png
Actions > Testes automatizados > execução: os dois jobs verdes (Testes automatizados e Build da imagem Docker).

Figura - Execução do workflow no GitHub Actions

PRINT PENDENTE: docs/prints/ci-02-resumo.png
Página da execução, aba Summary: tabelas de totais, cenários BDD e testes de API.

Figura - Resumo da execução (Job Summary)

PRINT PENDENTE: docs/prints/ci-03-passo-bdd.png
Job Testes automatizados > passo 2 expandido, mostrando os cenários do Cucumber e 'Tests run: 69'.

Figura - Log do passo de cenários BDD e testes de API

PRINT PENDENTE: docs/prints/ci-04-relatorio-cucumber.png
Artefato relatorio-cucumber > cucumber.html aberto no navegador: 100% dos cenários aprovados.

Figura - Relatório do Cucumber gerado no pipeline

9. Bug encontrado e desafios

9.1 Bug real encontrado pelos testes

Problema. Na primeira execução, os testes de API mostraram que o mapa `erros` das respostas de validação usava o nome do atributo Java (`populacaoAtendida`, `equipamentoId`) em vez do nome do campo no JSON (`populacao_atendida`, `equipamento_id`). O cliente enviava um campo e o erro apontava para outro, inexistente no contrato da API.

Correção. O `TratadorDeErros` passou a traduzir o nome com a mesma estratégia de nomes do Jackson, e um teste unitário novo (`errosDeValidacaoUsamOsNomesDeCampoDoJson`) impede a regressão.

Listagem - correção em TratadorDeErros.java

```
    private String nomeNoJson(String campoJava) {
        return json.getPropertyNamingStrategy() instanceof PropertyNamingStrategies.NamingBase
estrategia
                ? estrategia.translate(campoJava)
                : campoJava;
    }
```

9.2 Desafios e soluções

Desafio	Solução
Contrato assumia que todo beneficiário tinha protocolo BNF-; os dados reais têm também turmas (TRM-6A-ALZ)	Contrato revisado com base nos dados: qualquer pseudônimo no formato XXX-..., mantendo proibidos nome, CPF, RG, e-mail e telefone
Cucumber 7.34 é compilado para JUnit 5.14, e o Spring Boot 3.5 traz o 5.12	Versão do JUnit alinhada no pom.xml (junit-jupiter.version), evitando erros de compatibilidade da JUnit Platform
O conversor de inteiros do Cucumber lê "09" (setembro) como número octal	Data do cenário lida como texto e convertida com DateTimeFormatter (dd/MM/yyyy)
REST Assured lê decimais do JSON como Float, e 83702.0 não "batia" com o Double esperado	Configuração global do REST Assured para ler decimais como Double
Estação de desenvolvimento sem Docker	Modo duplo: Testcontainers no pipeline e MongoDB local via TEST_MONGODB_URI, com trava que exige um banco descartável
Resultados que dependem da data de execução (licenças vencidas)	Relógio de teste injetado na aplicação, com data fixa e ajustável por cenário
Testes que interferem uns nos outros	Restauração da carga inicial antes de cada cenário e de cada teste
Acentos dos cenários ilegíveis no console do Windows	Somente visual: relatórios em UTF-8 corretos; README orienta usar chcp 65001

10. Tecnologias e atendimento aos requisitos

Camada	Tecnologias
Aplicação	Java 21, Spring Boot 3.5 (Web, Validation, Data MongoDB, Actuator), springdoc-openapi
Banco de dados	MongoDB 8.0
BDD	Cucumber 7 (cucumber-java, cucumber-spring, cucumber-junit-platform-engine), Gherkin em português
Testes de API	REST Assured 5, REST Assured JSON Schema Validator (draft-04)
Base de testes	JUnit 5 + JUnit Platform Suite, Spring Boot Test, Testcontainers, Mockito, AssertJ/Hamcrest
Cobertura	JaCoCo
Build e CI	Maven Wrapper, GitHub Actions, Docker, Docker Compose

10.1 Atendimento aos requisitos

Requisito	Como foi atendido
No mínimo 3 cenários BDD em Gherkin, positivos e negativos	5 funcionalidades e 17 cenários escritos (23 executados), sendo 9 de falha, todos em Gherkin (seção 3)
Testes de API para todas as APIs	11 rotas /api e as APIs de plataforma; cobertura total verificada contra o OpenAPI (seção 4)
Validação de status code	Todos os testes de API e cenários BDD verificam o status HTTP
Validação do corpo de resposta (JSON)	Valores, quantidades, ordenação e efeitos colaterais verificados com REST Assured
Testes de contrato com JSON Schema	17 contratos em src/test/resources/schemas, aplicados a todas as respostas
Execução simples e documentada, local ou em CI	./mvnw verify (com Docker ou MongoDB local) e workflow do GitHub Actions; README e seção 6
Código-fonte em .zip com testes, Dockerfiles, scripts e README	Pacote gerado por tools/empacotar-entrega.ps1
Documentação com cenários Gherkin, configurações e evidências	Este documento (seções 3, 5, 7 e 8)